COMPARATIVA DE ESTRUCTURAS DE DATOS PRIMARIAS Y SECUNDARIAS

Estructura de Datos	**Tipo de Acceso / Principio**	**Descripc ión Corta**	**Operacio nes Principal es (Primitiv as)**	**Operacio nes Ocasion ales / Funcion es Auxiliare s**	**Compleji dad Temporal (Promedi o)**	**Casos de Uso Principal es**
Arreglo (Array)	Acceso Directo (Índice)	Colección contigua de elemento s del mismo tipo con tamaño fijo.	get(i), set(i, val)	length(), search(va l), sort(), reverse(), slice	Acceso: O(1) Búsqued a: O(n) Inserción/ Borrado: O(n)	Tablas, vectores, acceso rápido por posición conocida.
Arreglo Dinámic o (Dynami c Array / List)	Acceso Directo (Índice)	Arreglo que redimensi ona su capacida d automátic amente al llenarse.	append(v al), get(i), set(i, val), remove(i)	capacity() , resize(), indexOf(v al), extend(), clear()	Acceso: O(1) Inserción al final: O(1) amortizad o Inserción/ Borrado	Listas generales en Python/J ava, acumulad ores de tamaño variable.
Pila (Stack)	LIFO (Last In, First Out)	Estructur a donde el último elemento en entrar es el primero en salir.	push(val), pop()	peek() / top(), isEmpty() , isFull(), size(), clear()	Inserción (Push): O(1) Eliminaci ón (Pop): O(1) Búsqued a: O(n)	Pila de llamadas de funciones (Recursió n), deshacer/

						rehacer (Ctrl+Z), evaluació n de expresion es (Sintaxis) .
Cola (Queue)	FIFO (First In, First Out)	Estructur a donde el primer elemento en entrar es el primero en salir.	enqueue(val), dequeue( )	front() / peek(), isEmpty() , isFull(), size(), rear()	Inserción (Enqueue): O(1) Eliminaci ón (Dequeue): O(1) Búsqued a: O(n)	Colas de impresión , procesam iento de tareas en segundo plano, buffers de datos.
Cola de Doble Extremo (Deque)	LIFO + FIFO (Ambos extremos)	Permite insertar y eliminar elemento s tanto por el frente como por la parte trasera.	push_fro nt(val), push_bac k(val), pop_front (), pop_back ()	peek_fro nt(), peek_bac k(), isEmpty() , rotate(), clear()	Inserción en extremos: O(1) Eliminaci ón en extremos: O(1) Búsqued a: O(n)	Algoritmo s de ventana deslizant e, historial de navegaci ón bidireccio nal.
Tabla Hash (Hash Table / Dictionar y)	Clave - Valor (Hashing)	Estructur a que asocia claves a valores usando una función hash.	insert(key , val), get(key), delete(ke y)	containsK ey(key), keys(), values(), rehash(), hashFunc tion(key)	Acceso: O(1) Inserción: O(1) Eliminaci ón: O(1) Peor caso (colisione s): O(n)	Bases de datos, indexació n rápida, cachés, diccionari os, conteo de frecuenci as

Lista Enlazada Simplemente (Singly Linked List)	Acceso Secuencial	Nodos dispersos en memoria unidos por punteros hacia el siguiente elemento.	insert_head(val), delete_head(), traverse()	insert_after(node, val),search(val), reverse(), length()	Acceso: O(n) Inserción al inicio: O(1) Inserción al final: O(n) o O(1) con tail	Gestión dinámica de memoria, implementación de pilas y colas simples.
Lista Doblemente Enlazada (Doubly Linked List)	Acceso Secuencial Bidireccional	Nodos con enlaces hacia el elemento siguiente y el elemento anterior.	insert_head(val), insert_tail(val), delete(node)	traverse_forward(), traverse_backward(), search(val)	Acceso: O(n) Inserción/ Eliminación con nodo conocido: O(1)	Sistemas de ficheros, listas de reproducción de música, caché LRU.
Conjunto Hash (HashSet)	Clave Única (Sin Duplicados)	Colección no ordenada de elementos únicos respaldada por una Tabla Hash.	add(val), remove(val), contains(val)	union(set B), intersection(setB), difference (setB), size()	Inserción: O(1) Eliminación: O(1) Búsqueda: O(1)	Eliminación de duplicados, prueba rápida de pertenencia de elementos